

Amazon ML Challenge 2026 — Team B Playbook

Focus Area: LightGBM Training from S3 Features, Macro F_{0.5} Optimization & Submissions

Operating Window: Friday 26th September (Day 1 Baseline) & Saturday 27th September (Full Optimization)

Target Milestone: Produce valid leaderboard submissions and final verified `<team_name>_submission.zip`.

1. Executive Summary & Team Mission

Team B takes the candidate feature parquets produced by Team A from the **central AWS S3 data store** and delivers the competitive results.

Compute is executed on **Google Colab or local machines** (no SageMaker notebook instances needed), pulling features directly from S3.

Your core mission:

- Day 1 Fast Baseline:** Pull the 50k sample feature parquet from `s3://<bucket>/features/sample_50k/`, train a baseline LightGBM model, and submit a 100% valid TSV to the Unstop portal by 3:00 PM on Sept 26.
- Train Gradient Boosting (LightGBM):** Fast, robust, handles class imbalance, requires no GPU.
- Optimize Decision Threshold for Macro $F_{0.5}$:** Standard 0.5 threshold fails because $F_{0.5}$ weights precision 2x over recall. Sweep to find the peak (typically 0.65 to 0.80).
- Master Singleton Logic:** Predict empty string for entities without strong matches (scores a full 1.0).
- Pass Official Validator:** Zero syntax rejections using `utils/validate_submission.py`.
- S3 Artifact Storage & Package Final Zip:** Save model weights and submission TSVs to `s3://<bucket>/submissions/` and package `<team_name>_submission.zip`.

2. Team B Member Roles & Schedule

Member Role	Primary Focus	Key Responsibilities
Member 3 (Model Specialist)	LightGBM & Metrics	Model hyperparameter tuning, scale_pos_weight, custom Macro F _{0.5} evaluator, S3 feature consumption
Member 4 (MLOps & Submission)	Formatting & Packaging	Test inference, threshold sweep, validator script, methodology document & zip packaging

Timeline for Sept 26 & 27

- Sept 26 (02:00 PM - 04:00 PM):** Pull 50k sample from `s3://<bucket>/features/sample_50k/`. Train baseline & submit **Day 1 Baseline** to Unstop.
- Sept 26 (04:00 PM - 08:00 PM):** Build and test the custom Macro $F_{0.5}$ metric script and threshold sweep logic.
- Sept 26 (10:00 PM):** Confirm Team A has uploaded the full parquets to `s3://<bucket>/features/full/`.
- Sept 27 (09:00 AM - 12:00 PM):** Train LightGBM on the full training dataset pulled from S3.
- Sept 27 (12:00 PM - 03:00 PM):** Optimal threshold tuning on validation set $\rightarrow$ Generate test predictions $\rightarrow$ Submit run 1.
- Sept 27 (03:00 PM - 07:00 PM):** Feature importance analysis, ensemble or hyperparameter tune $\rightarrow$ Submit run 2 & 3.
- Sept 27 (07:00 PM - 10:00 PM):** Final validation, fill `Documentation_template.md`, build `<team_name>_submission.zip`, final submit!

3. Step-by-Step Implementation Guide

Step 1: Ingest Features from S3 & Train LightGBM

Pull feature tables directly from S3 without local disk clutter:

```

import os
import pandas as pd
import lightgbm as lgb

# Credentials (from environment or Colab secrets)
storage_opts = {
    "key": os.environ.get("AWS_ACCESS_KEY_ID"),
    "secret": os.environ.get("AWS_SECRET_ACCESS_KEY")
}
BUCKET = "your-hackathon-bucket"

# 1. Load Parquet files directly from central S3
print("Loading feature sets from S3...")
train_df = pd.read_parquet(f"s3://{BUCKET}/features/full/train_candidates.parquet", storage_options=storage_opts)
val_df = pd.read_parquet(f"s3://{BUCKET}/features/full/val_candidates.parquet", storage_options=storage_opts)

features = [
    'name_ratio', 'name_partial', 'name_token_sort', 'name_token_set',
    'addr_ratio', 'addr_partial', 'addr_token_set',
    'name_len_diff', 'addr_len_diff'
]

X_train, y_train = train_df[features], train_df['label']
X_val, y_val = val_df[features], val_df['label']

# 2. Configure LightGBM Classifier with class imbalance handling
model = lgb.LGBMClassifier(
    n_estimators=800,
    learning_rate=0.05,
    num_leaves=31,
    subsample=0.8,
    colsample_bytree=0.8,
    scale_pos_weight=3.5,
    random_state=42
)

model.fit(
    X_train, y_train,
    eval_set=[(X_val, y_val)],
    callbacks=[lgb.early_stopping(stopping_rounds=50)]
)

# 3. Save model weights to central S3
import joblib
joblib.dump(model, "lgbm_model.pkl")
# Push model to S3
import boto3
s3 = boto3.client('s3', aws_access_key_id=storage_opts['key'], aws_secret_access_key=storage_opts['secret'])
s3.upload_file("lgbm_model.pkl", BUCKET, "models/lgbm_model_latest.pkl")
print("✅ Model weights synced to S3.")

```

Step 2: Implement Macro $F_{0.5}$ & Sweep Thresholds

The competition evaluates per-Source-1 macro-average $F_{0.5}$. False merges are penalized heavily (precision matters 2x more than recall):

```
import numpy as np

def compute_f05(precision, recall):
    if (0.25 * precision + recall) == 0:
        return 0.0
    return (1.25 * precision * recall) / (0.25 * precision + recall)

val_probs = model.predict_proba(X_val)[: , 1]

best_threshold = 0.50
best_f05 = 0.0

# Sweep threshold from 0.40 to 0.92
for t in np.arange(0.40, 0.92, 0.04):
    preds = (val_probs >= t).astype(int)
    tp = np.sum((preds == 1) & (y_val == 1))
    fp = np.sum((preds == 1) & (y_val == 0))
    fn = np.sum((preds == 0) & (y_val == 1))

    prec = tp / (tp + fp) if (tp + fp) > 0 else 0
    rec = tp / (tp + fn) if (tp + fn) > 0 else 0
    score = compute_f05(prec, rec)

    print(f"Threshold: {t:.2f} | Precision: {prec:.3f} | Recall: {rec:.3f} | F_0.5: {score:.4f}")
    if score > best_f05:
        best_f05 = score
        best_threshold = t

print(f"\nWINNING THRESHOLD: {best_threshold:.2f} (Validation F_0.5 = {best_f05:.4f})")
```

Step 3: Format Test Outputs (All 1,732,544 S1 Entities)

You must generate `output/matching_results.tsv` and `output/candidate_pairs.tsv`. Every single entity in `test_source1.tsv` must have exactly one row:

```

# Predict probabilities for test candidate pairs loaded from S3
test_df = pd.read_parquet(f"s3://{BUCKET}/features/full/test_candidates.parquet", storage_options=storage_opts)
test_df['prob'] = model.predict_proba(test_df[features])[:, 1]
test_df['is_match'] = test_df['prob'] >= best_threshold

# Group matches and candidates
matches_map = (
    test_df[test_df['is_match']]
    .groupby('source1_entity_id')['candidate_entity_id']
    .apply(lambda ids: ", ".join(list(dict.fromkeys(ids))))
    .to_dict()
)

candidates_map = (
    test_df
    .groupby('source1_entity_id')['candidate_entity_id']
    .apply(lambda ids: ", ".join(list(dict.fromkeys(ids))))
    .to_dict()
)

# Read all required test S1 IDs (from S3 raw or local)
test_s1 = pd.read_csv(f"s3://{BUCKET}/raw/test_source1.tsv", sep="\t", storage_options=storage_opts)
all_s1_ids = test_s1['entity_id'].tolist()

matching_rows = []
candidate_rows = []

for s1_id in all_s1_ids:
    matching_rows.append({
        'source1_entity_id': s1_id,
        'matched_entity_ids': matches_map.get(s1_id, "")
    })
    candidate_rows.append({
        'source1_entity_id': s1_id,
        'candidate_entity_ids': candidates_map.get(s1_id, "")
    })

os.makedirs("output", exist_ok=True)
pd.DataFrame(matching_rows).to_csv("output/matching_results.tsv", sep="\t", index=False)
pd.DataFrame(candidate_rows).to_csv("output/candidate_pairs.tsv", sep="\t", index=False)
print("TSV files created successfully!")

# Backup submission directly to S3
s3.upload_file("output/matching_results.tsv", BUCKET, "submissions/matching_results_latest.tsv")

```

Step 4: Validate Before Submitting (Zero Rejections)

Run this check locally:

```
python3 utils/validate_submission.py \  
  --matching output/matching_results.tsv \  
  --candidate output/candidate_pairs.tsv \  
  --test-dir dataset/test
```

Only upload to the portal when it prints: **PASS – no blocking issues found. Safe to submit.**

Step 5: Final Submission Package Assembly

Before the deadline, build the final zip:

```
zip -r my_team_submission.zip \  
  output/matching_results.tsv \  
  output/candidate_pairs.tsv \  
  src/ \  
  docs/Documentation_template.md  
  
# Upload final zip to S3 backup  
s3.upload_file("my_team_submission.zip", BUCKET, "submissions/final_team_submission.zip")
```

4. Team B Golden Rules & Anti-Mistakes

- **DO NOT** use default 0.5 threshold. $F_{0.5}$ rewards precision; your optimal threshold will usually be 0.65–0.78.
- **DO NOT** miss any S1 entity. All 1,732,544 IDs from `test_source1.tsv` must appear.
- **DO NOT** submit without running `utils/validate_submission.py` locally.
- **DO NOT** upload a zipped file directly to the leaderboard portal. The portal takes only `matching_results.tsv`. The zip is for final code review.